

Contents

CareerOS Frontend Codebase Analysis & Production Readiness Report	1
1. Executive Summary	1
2. Project Overview	1
3. Feature Inventory	3
4. Code Quality Analysis	5
5. Bug & Risk Inspection	7
6. Backend & Database Readiness	10
7. Production Readiness Review	13
8. Prioritized Recommendations	14
9. Commands & Check Results	18
10. Appendix	18

CareerOS Frontend Codebase Analysis & Production Readiness Report

Project: Career OS — candidate-centered career intelligence platform with a parallel employer marketplace **Repo path:** C:\Users\Ethan\Desktop\career-os **Analyst:** Claude (Cowork mode, automated codebase review) **Date:** 2026-05-26 **Scope:** Frontend only. No backend exists yet. All persistence is in-browser (localStorage).

1. Executive Summary

Career OS is a Next.js 16 / React 19 / Tailwind v4 SPA that simulates a two-sided “career intelligence” product: candidates onboard, design their life chapters, build a Living Portfolio, and watch a hidden growth loop animate readiness toward target jobs; employers onboard, browse a marketplace of mock candidates, save, invite, chat, and receive simulated notifications. The visual design system is mature and disciplined (semantic luminous / clover / brand token contract, glass surface tiers, 12-col grid). All “logic” is mock or simulated — there is no backend, no auth, no real-time, no matching algorithm beyond static scores.

Bottom line: - **Visual + UX prototype: production-quality.** Clean, responsive, accessible at a basic level, consistent token usage. - **Data + logic: prototype-only.** Every score, match, notification, message, and persisted state lives in the browser. - **TypeScript source is clean** (tsc --noEmit on src/ passes with 0 errors). One legitimate lint pattern repeats 6 times across context providers (localStorage hydration triggers React 19’s react-hooks/set-state-in-effect rule). - **Build cannot be verified offline** (Next.js requires fetching @next/swc-* from npm). - **Repository hygiene needs cleaning** — 18 leftover .tmp/.nonulls/.clean files in src/ from the null-byte scrub workflow documented in CLAUDE.md §8. - **Big gaps before production:** no loading.tsx, no error.tsx, no not-found.tsx, no real form validation, no auth, no API surface, no test harness.

2. Project Overview

2.1 What this product does

Career OS is a candidate-centered career intelligence platform. The candidate experience is built around a “hidden growth loop”: user signals (skills, life events, target jobs) feed a Living Portfolio that compounds into a readiness trajectory and surfaces opportunities. The employer experience is a parallel mini-platform: hiring goal in → marketplace of curated candidates out, with invite / save / chat affordances.

2.2 Stack & conventions

Item	Value
Framework	Next.js 16.2.6 (Turbopack, App Router)
UI	React 19.2.4, TypeScript strict
Styling	Tailwind v4 (@tailwindcss/postcss), forced dark mode
Icons	lucide-react
State	React contexts + localStorage; no Redux / Zustand / Query
Persistence	localStorage only
Auth	None
Testing	None configured
Lint	eslint + eslint-config-next

2.3 Main user flows

1. **Candidate flow.** Landing (/) → onboarding (/onboarding) → dashboard (/candidate/dashboard) with side pages (/candidate/portfolio, /candidate/chapters). Adding a skill triggers an 8-step animated growth loop and fires notifications.
2. **Employer flow.** Landing → onboarding (/employers/onboarding) → analyzing screen → marketplace (/employers/marketplace) → filter / save / view profile (/candidates/[id]) / invite (/employers/contact/[id]) → success → simulated “accepted” notification → chat (/employers/chat/[id]).
3. **Cross-flow.** Public candidate profile route is open and intentionally lives outside the employer shell so it can be shared.

2.4 Routes (12 in total)

Route	File	Purpose
/	src/app/page.tsx	Marketing landing, dual CTAs
/onboarding	src/app/onboarding/page.tsx	Candidate 5-step intent capture
/candidate/dashboard	src/app/candidate/dashboard/page.tsx	Growth loop, skill evolution, readiness
/candidate/portfolio	src/app/candidate/portfolio/page.tsx	Living Portfolio CV builder
/candidate/chapters	src/app/candidate/chapters/page.tsx	Chapter Designer (weekly/monthly/yearly)
/candidates/[candidateId]	src/app/candidates/[candidateId]/page.tsx	Public candidate profile (read-only mock)
/employers/onboarding	src/app/employers/onboarding/page.tsx	3-questions hiring intent + analyzing
/employers/marketplace	src/app/employers/marketplace/page.tsx	Candidate cards, search, filters, detail modal
/employers/saved	src/app/employers/saved/page.tsx	Shortlist
/employers/notifications	src/app/employers/notifications/page.tsx	Employer-side notification feed
/employers/contact/[candidateId]	src/app/employers/contact/[candidateId]/page.tsx	Invite detail, 1-page composer
/employers/chat/[candidateId]	src/app/employers/chat/[candidateId]/page.tsx	Private 1:1 prototype

2.5 Architecture map

- **App routes:** src/app/<route>/page.tsx — App Router, mostly Client Components ("use client" at the top of nearly every page).

- **Shells:** AppShell (candidate) and EmployerAppShell (employer) — parallel, do not share code beyond the <LayoutLines/> background.
- **Top navigation:** TopMenu (candidate, dropdown) and EmployerTopMenu (employer, inline + mobile dropdown).
- **Notification surfaces:** NotificationBell (candidate, listens to NotificationsContext) and EmployerNotificationsBell (employer, listens to SavedCandidatesContext).
- **Contexts (src/app/layout.tsx providers stack):** IntentProvider → NotificationProvider → PortfolioProvider → ChaptersProvider → EmployerProvider → SavedCandidatesProvider. All localStorage-backed.
- **Domain UI:** src/components/{dashboard,portfolio,chapters,intent-form,marketplace,employer,onboarding,marketing,ui,shared,app}.
- **Data:** src/lib/candidates/data.ts (17 mock candidates), src/lib/jobs/destinations.ts (4 mock destinations + skill-match helpers), src/lib/chapters/data.ts (chapter types + helpers), src/lib/portfolio/data.ts (legacy section types), src/components/intent-form/data.ts (stages/goals/fields/target jobs).
- **Design tokens:** src/app/globals.css (oklch CSS variables, glass-N utilities, line utilities, appear/appear-zoom keyframes) — Tailwind v4 via @theme inline.

3. Feature Inventory

#	Feature	Files	Status	Storage / Data	Notes
1	Marketing landing	src/app/page.tsx src/components/quality-viewing/*	Production-ready	Static	Three “Find Talent” CTAs all route to /employers/onboarding; one in-page anchor #candidate-vp remains in the hero badge
2	Candidate onboarding (5 visible steps)	src/app/onboarding/page.tsx src/components/onboarding-form/*	Functional prototype	localStorage (career-os-intent)	Stage → Goals → Details → Target job → Analyzing → Complete
3	Candidate dashboard + growth loop	src/app/candidate-dashboard/page.tsx src/components/candidate-dashboard/*	Functional prototype	localStorage component state	8-step state machine in useGrowthLoop.ts; no remote sync
4	Living Portfolio (CV builder)	src/app/candidate-portfolio/page.tsx src/components/candidate-portfolio/{PortfolioBuilder,CVPreview}.tsx src/lib/hooks/usePortfolio.tsx	Functional prototype	localStorage	Has legacy PortfolioPreview.tsx shim file PortfolioPreview.tsx re-exporting CVPreview

#	Feature	Files	Status	Storage / Data	Notes
5	Life Chapter Designer	src/app/candidates/pageStorage src/components/prototypes/ src/lib/context/ChaptersContext.tsx	Functional prototype	pageStorage (career-os-events)	Weekly/Monthly/Yearly views; Design Mode opens EventEditor
6	Candidate notifications	src/components/functional/notifications src/lib/context/notifications.tsx	Functional prototype	local storage (persisted)	Fires on skill-add and on full job-match coverage
7	Job-match engine (mock)	src/lib/jobs/destinations.ts	Functional prototype	4 hardcoded destinations	Case-insensitive partial substring matcher
8	Employer onboarding	src/app/employers/pageStorage src/components/prototypes-onboarding/*	Functional prototype	local storage (career-os-employer-goal, career-os-employer-onboarded)	3 questions + analyzing screen + completion
9	Employer marketplace	src/app/employers/pageStorage src/components/prototypes-marketplace/*	Functional prototype	local storage (17 mocks)	Search, data category chips + detail modal; sorts by static matchScore
10	Candidate detail modal	src/components/functional/mocks/CandidateDetailModal.tsx	Functional prototype	Mock	Outside, Esc, scroll-lock, modal accessibility correct
11	Public candidate profile	src/app/candidates/pageStorage src/lib/context/candidates/*	Functional prototype	Mock	Renders from findCandidateById; no shell so it's publicly shareable
12	Saved candidates	src/app/employers/pageStorage src/components/prototypes-savedCandidates/* src/lib/context/SavedCandidatesContext.tsx	Functional prototype	local storage (saved)	Empty state + remove from saved icon
13	Invite / contact composer	src/app/employers/pageStorage src/components/prototypes-contactForm/*	Functional prototype	local storage (employer-invited)	Page fills subject/template; success modal; 3.5 s simulated acceptance
14	Employer notifications	src/app/employers/pageStorage src/components/prototypes-employer-notifications/*	Functional prototype	local storage (employer-notifications)	Empty state; links to chat for accepted invites

#	Feature	Files	Status	Storage / Data	Notes
15	Private chat	src/app/employer/[candidateId]/page.tsx src/components/private/ChatInterface.tsx	Functional / prototype	localStorage (CandidateInterface.messages; employer-chats)	Seed, 3 mock messages; 1.4 s simulated reply
16	Reset / start over	AppShell, Employer-TopMenu	Functional	localStorage clears	Wipes the relevant slice; not destructive across roles
17	Parallax + features carousel	src/components/Producing/{Parallax, FeaturesCarousel}.tsx	Producing / quality visual	Static	preserves reduced-motion respected

3.1 Hidden / non-obvious features

- **Job-match auto-firing.** `src/app/candidate/dashboard/page.tsx` watches `state.skills` and fires one important notification the first time the user covers every required skill for any destination in `DESTINATIONS`.
- **Notification dedupe.** Dashboard uses a `useRef<Set<string>>(...)` to ensure a given match fires only once per session (CLAUDE.md §9 calls this out explicitly).
- **Cascading status sliver.** `PageHeader` ships a `loopStatus` prop that surfaces “Cascading · step X of 8” while `useGrowthLoop` is running.
- **Skill input sanitization.** `useGrowthLoop.ts` strips quotes (`"`, `"`, `"`) and caps length at 60 to keep cards intact and avoid breaking the AI-recommendation template that includes the skill in a string.
- **Auto-scroll chat.** `ChatInterface` auto-scrolls when message count changes.

4. Code Quality Analysis

4.1 Component structure

- **Single responsibility is largely respected.** Pages compose shells, page headers, grid, and feature cards. Most cards are 100–300 LOC, readable.
- **Two parallel shells.** `AppShell` (candidate) and `EmployerAppShell` (employer) avoid forking concerns. Per the spec this was the right call.
- **Reuse:** `StepShell` is shared between candidate and employer onboarding. `Button`, `LinkButton`, `Badge`, `LayoutLines`, `Glow` are reused across surfaces.

4.2 State management

- **6 context providers** stacked in `src/app/layout.tsx`. Each is its own concern and each hydrates from `localStorage` in a mount effect.
- **No state-management library** — fine for this scale.
- **No optimistic UI / cache layer** — fine because nothing is networked yet.
- **One historic risk** (CLAUDE.md §9) is `setState` calls fired inside another component’s `setState` updater; the code consistently uses `useEffect` deferral. `NotificationBell` and `EmployerNotificationsBell` both defer `markAllRead` to a `useEffect(... [isOpen, markAllRead])`.

4.3 Routing

- **App Router**, all routes use `page.tsx`. Three dynamic routes correctly type params as `Promise<{ ... }>` and unwrap with `use(params)` (matches Next.js 16 conventions in `node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/dynamic-routes.md`).
- **No `loading.tsx` / `error.tsx` / `not-found.tsx`** in any route segment. `notFound()` is called in the three dynamic routes when a candidate id is invalid, but there is no custom 404 page — users will see Next's default.

4.4 Data flow

- Mostly one-way: contexts → pages → cards. Cards receive callbacks (`onSaveToggle`, `onOpenDetails`) and never reach into context directly. The marketplace page is the orchestrator.
- **Effect ↔ localStorage round-trip is consistent across all five context providers.** Hydration sets `isHydrated`; writes are gated on `isHydrated`. Reads only happen in browser. No SSR mismatch.

4.5 TypeScript usage

- **Strict mode on**, `noEmit: true` in `tsconfig` (Next.js handles emission).
- **Public types are explicit and reused** — `Candidate`, `EmployerGoal`, `ChatMessage`, `LoopStep`, `Intent`, etc.
- **any is essentially absent.** I found no `: any` in `src/` outside type-narrow as casts. The only unknown is `payload?: Record<string, unknown>` in `AppNotification`.

4.6 Styling consistency

- **Excellent token discipline.** Every interactive surface uses semantic ramps (`luminous/clover/brand`) per the contract in `src/lib/themes.ts` lines 1–37.
- **No `bg-gray-XXX` in product UI.** The legacy `appTheme` literal mapping in `lib/themes.ts` (lines 44–94) is **not imported anywhere** — it's dead code.
- **Glass tiers** (`glass-3/glass-4`) used per the contract in `CLAUDE.md` §4.

4.7 Responsiveness

- **All pages** wrap content in `max-w-container mx-auto` and use `Grid12/Col` or grid utilities.
- **Marketplace** collapses 3 → 2 → 1 column; nav becomes hamburger ≤ md; modal sizes to viewport.
- **Chat composer** uses standard form layout, no horizontal overflow risk found.

4.8 Accessibility basics

- **Modals** (`CandidateDetailModal`, `ContactForm success modal`, `ConfirmDialog`, `NotificationBell.ReachOutModal`) all set `role="dialog"`, `aria-modal="true"`, `aria-labelledby`, listen for Escape, and lock body scroll where appropriate.
- **Bells & menus** use `aria-haspopup`, `aria-expanded`, `focus-visible` rings.
- **Score chart bars** render with no `role/aria-valuenow` — minor ally gap for screen readers.
- **Color contrast** is dictated by `oklch` tokens; not formally verified in this analysis.

4.9 Animation / performance

- **CSS keyframes only** (`appear`, `appear-zoom`) — no JS animation libraries, matches the constraint in `CLAUDE.md` §5.
- **Parallax** uses `requestAnimationFrame` + `prefers-reduced-motion` opt-out.

- **useGrowthLoop** uses a single setTimeout chain. No leaks identified: a useEffect(() => () => clearTimeout(timeoutRef.current)) cleans up on unmount.
- **No expensive renders identified.** The marketplace memoizes the filtered list with useMemo.

4.10 Naming + separation of concerns

- Naming is consistent: PascalCase components, camelCase hooks/utilities, [candidateId] for dynamic route param everywhere.
- File layout matches CLAUDE.md §3 perfectly.

4.11 Maintainability concerns

- The 18 leftover .tmp/.nonulls/.clean files in src/ create noise during search (Grep returns duplicate matches).
- PortfolioPreview.tsx is a 5-line shim re-exporting CVPReview. DashboardShell.tsx is a 5-line shim re-exporting AppShell. Both compile but pollute the search/import surface.
- lib/themes.ts ships 156 lines, of which only themes (used by intent flow? no) and semantic/appTheme are conceptually documented but **not imported by any file**.

5. Bug & Risk Inspection

5.1 Routing / dead links

Severity	Location	Issue
Low	src/app/page.tsx:128	Hero badge anchors to #candidate-vp — works, harmless, but it's a hash anchor not a route
Low	src/components/app/TopMenu.tsx:155	MenuButtonStub “Engage with Employers” labeled soon — does nothing. Now that the employer flow exists, this stub could route to /employers/onboarding
Info	src/components/marketing/Navbar.tsx:48	Sign in link to /candidate/dashboard — there is no auth, so this drops the user straight into the dashboard if they have intent, or redirects to /onboarding if they don't

5.2 Missing pages / states

Severity	Issue	Files needed
High	No not-found.tsx for /candidates/[id], /employers/contact/[id], /employers/chat/[id]	Calls notFound() which renders Next.js's default page
High	No error.tsx boundaries anywhere	An uncaught error becomes a Next.js full-screen error

Severity	Issue	Files needed
Medium	No loading.tsx anywhere	All pages render synchronously today but this will matter once data fetching is added

5.3 Mock-data hazards

Severity	Issue
High	All 17 candidates in src/lib/candidates/data.ts are fictional. The marketplace currently looks like a real product but every match score / readiness / growth signal is hard-coded. Risk of demo confusion if shipped without a "Demo data" badge.
High	Notifications are simulated via setTimeout(... 3500) in ContactForm.tsx. Looks realistic; could mislead stakeholders.
Medium	Job match scores are static (match: 88 etc.) — countMatches/coverage are real, but the per-destination static match field overrides anything dynamic the UI might be expected to compute.

5.4 Hardcoded values

Severity	Location	Value	Should become
Medium	src/lib/candidates/data.ts	17 candidates	API call GET /api/candidates?...
Medium	src/lib/jobs/destinationList	4 destinations + skill list	API
Medium	src/components/intentForm/data.ts	5 stages, goals, fields, target jobs	API or CMS
Low	src/components/marketplace/HiringGoalSummary	"Banco HiringGoalSummary Southeast Asia" fallback	Backend-derived
Low	src/components/marketplace/EmailSubject	Default/EmailSubject	Server-side template + body

5.5 Duplicated / dead code

Severity	Issue
Medium	src/components/dashboard/DashboardShell.tsx is a 5-line shim re-exporting AppShell, no callers found in src/
Medium	src/components/portfolio/PortfolioPreview.tsx is a 5-line shim re-exporting CVPreview, no callers found in src/

Severity	Issue
Medium	src/lib/themes.ts exports appTheme, semantic, themes — none are imported. Comment block at the top is the only useful artifact (the color contract).
Medium	18 .tmp / .nonulls / .clean debris files in src/ from the null-byte scrub workflow
Low	Two empty .tsconfig.*.json files at the project root left over from this analysis run (the workspace permission layer prevented rm; they are valid empty JSON {})

5.6 Fragile state / hydration

Severity	Location	Issue
Low	All 5 context providers	setState inside the hydration useEffect produces ESLint react-hooks/set-state-in-effect errors (6 in total). The pattern is correct for SSR-safe localStorage hydration, but the lint rule (new in React 19's strict ruleset) flags it.
Low	NotificationsContext is not persisted to localStorage; only the employer-side SavedCandidatesContext notifications persist. A page refresh on the candidate dashboard loses all in-session notifications.	
Low	EmployerAppShell returns null until hydration completes — fine, but produces a brief blank flash. A loading.tsx would smooth this.	

5.7 Form handling / validation

Severity	Issue
Medium	ContactForm only checks !subject \\ \\ !message (truthy check). No email-style validation, no length limits, no XSS sanitization.
Medium	RoleStep (employer-onboarding) does value.trim().length > 1 — minimal but reasonable.

Severity	Issue
Medium	SkillEvolutionCard accepts any text up to 60 chars; only quotes are stripped. Profanity / PII not filtered.
Medium	No form library; everything is hand-rolled. Migrating to React Hook Form + Zod would centralize validation.

5.8 Loading / error states

Severity	Issue
Medium	CVPReview shows "Loading CV..." on !isHydrated. Most other pages just render against the initial empty state — fine because there's no network.
High	No error states exist anywhere. Once APIs are wired in, every fetch site needs error handling.

5.9 TypeScript / lint / build

- **TypeScript:** `tsc --noEmit` on `src/` exits 0. The two `.next/dev/types/` errors (stale truncated Next.js dev artifacts) clear after a fresh `next dev` or `next build`.
- **ESLint:** 6 errors, all `react-hooks/set-state-in-effect`. Files: `IntentContext.tsx`, `EmployerContext.tsx`, `SavedCandidatesContext.tsx`, `ChaptersContext.tsx`, `usePortfolio.tsx`. **Fixable** but not urgent; the pattern is intentionally SSR-safe.
- **Build:** `next build` cannot be verified — SWC binary fetch from `registry.npmjs.org` fails in the sandbox (CLAUDE.md §7 notes this). User must run `npm run build` locally.

5.10 Unused files

Path	Reason
<code>src/components/dashboard/DashboardShell.tsx</code>	Legacy shim, no callers
<code>src/components/portfolio/PortfolioPreview.tsx</code>	Legacy shim, no callers
<code>src/lib/themes.ts</code> (exports)	Constants not imported
<code>18 × *.tmp / *.nonnulls / *.clean</code>	Scrub debris
<code>.tsconfig.check.json / .tsconfig.src.json</code>	Empty {} analyzer scratch files

6. Backend & Database Readiness

6.1 What is currently mocked

Data	Source	Persisted in
Candidate pool (17)	<code>src/lib/candidates/data.ts</code>	n/a (constant)
Job destinations (4)	<code>src/lib/jobs/destinations.ts</code>	n/a
Onboarding stages/goals/fields/target jobs	<code>src/components/intent-form/data.ts</code>	n/a

Data	Source	Persisted in
Candidate intent	local state → career-os-intent	
Portfolio	local state → career-os-portfolio	
Chapters / events	local state → career-os-events	
Candidate notifications	local state, not persisted	
Employer goal	local state → career-os-employer-goal	
Employer onboarding flag	local state → career-os- employer-onboarded	
Saved candidate IDs	local state → career-os-employer-saved	
Invited candidate IDs	local state → career-os- employer-invited	
Employer notifications	local state → career-os- employer-notifications	
Chats	local state → career-os-employer-chats	

6.2 Likely API endpoints

Endpoint	Method	Purpose
/api/auth/login, /api/auth/logout, /api/auth/me	POST/GET	Auth + session
/api/candidate/intent	GET/PUT	Read/write the candidate's onboarding intent
/api/candidate/portfolio	GET/PUT	CRUD for portfolio sections
/api/candidate/chapters (or /events)	GET/POST/PATCH/DELETE	Life events CRUD
/api/candidate/notifications	GET, POST /read, POST /dismiss	Notification feed
/api/candidate/match	GET	Skill ↔ destination match coverage
/api/employer/goal	GET/PUT	Hiring goal
/api/employer/candidates?category=&q=&priorities=...	GET	Marketplace search
/api/employer/candidates/:id	GET	Detail
/api/employer/saved	GET/POST/DELETE	Save/unsave
/api/employer/invites	POST	Create invite
/api/employer/invites/:id/accept	POST (candidate-side)	Accept
/api/employer/notifications	GET, POST /read, POST /dismiss	Employer notifications
/api/employer/chats/:candidateId	GET	Conversation history
/api/employer/chats/:candidateId/messages	POST	Send message
/api/employer/chats/:candidateId/stream	WS, SSE	Real-time messages

6.3 Suggested database entities

```
users(id, email_hash, password_hash, role['candidate'|'employer'], created_at, last_seen_at)
candidates(
```

```

    id PK, user_id FK → users.id, name, stage, field, target_job, location,
    availability, headline, career_direction, target_role, industry, category,
    growth_signal, created_at, updated_at
)

candidate_skills(candidate_id FK, skill, source, added_at)
candidate_portfolio_items(id, candidate_id FK, kind['certificate'|'award'|'project'|'experience'],
candidate_chapters(id, candidate_id FK, kind, name, priority, scheduled_date, scheduled_time, s
candidate_match_scores(candidate_id FK, target_role, match_score, readiness_score, computed_at

employers(id PK, user_id FK → users.id, org_name, contact_name, created_at)
employer_goals(id, employer_id FK, talent_type, role, priorities TEXT[], location_pref, created
employer_saved_candidates(employer_id FK, candidate_id FK, saved_at, PK(employer_id, candidate
employer_invites(
    id, employer_id FK, candidate_id FK, subject, body,
    status ENUM('sent','accepted','declined','expired'),
    sent_at, responded_at
)

chats(id, employer_id FK, candidate_id FK, created_at, last_message_at, UNIQUE(employer_id, can
chat_messages(id, chat_id FK, sender ENUM('employer','candidate'), body, created_at)

notifications(
    id, recipient_user_id FK → users.id,
    kind ENUM('skill','job-match','invite-accepted','invite-pending','system'),
    severity ENUM('info','important'),
    title, body,
    payload JSONB,
    read_at, created_at, dismissed_at
)

job_destinations(id, title, company, required_skills TEXT[], match_score, is_active)

```

6.4 Auth requirements

- Two roles (candidate, employer). IntentContext.Role is already "candidate" | null; widen to "candidate" | "employer" | null and gate routes accordingly.
- Sign-in is a static placeholder in three locations: TopMenu, EmployerTopMenu, Navbar. Pick an auth provider (NextAuth / Clerk / custom OIDC) and gate /candidate/*, /employers/* behind middleware.
- Public surfaces stay open: /, /onboarding, /employers/onboarding, /candidates/[id] (intentionally public).

6.5 Risks when replacing local state with persistent data

Risk	Mitigation
Race conditions between optimistic UI + server writes	Use TanStack Query or useTransition + server actions; treat localStorage as a “draft” buffer
LocalStorage and DB disagree on schema	Version the LS payloads (v1, v2); on hydration, migrate or discard
Lost notifications during refresh (currently candidate notifications)	Move to notifications table; mark-read = UPDATE

Risk	Mitigation
Saved-IDs and invites currently use a <code>[id, ...]</code> list with no de-dupe at write time beyond <code>prev.includes(...)</code>	Add UNIQUE constraints at DB level
Chat seed data is generated on <code>ensureConversation</code>	Move seeding to server (or eliminate; replace with empty list + composer)
Mock <code>findCandidateById</code> is synchronous; UI assumes $O(1)$ lookup	Replace with <code>await fetch(...)</code> ; surface loading state per page

7. Production Readiness Review

7.1 What is production-ready

- Visual + interaction design across all flows
- Design tokens, glass surfaces, animation utilities
- Modal ally (focus, escape, scroll-lock)
- Hydration-safe `localStorage` pattern
- Strict TypeScript across `src/`
- Mobile-first responsive layout

7.2 What is prototype-only

- Every numeric score (match, readiness, growth signal)
- Every “notification” event — all are simulated locally
- Chat replies (`setTimeout(1400)` → fake candidate response)
- Marketplace ordering (static `matchScore` sort)
- Candidate pool (17 fixed records in `lib/candidates/data.ts`)
- “Sign in” buttons in three places

7.3 What should be fixed before deployment

- Add `loading.tsx`, `error.tsx`, `not-found.tsx` at minimum at the app root
- Add a “Demo data” banner (or remove mock data behind a feature flag) so stakeholders/users don’t mistake mocks for real listings
- Fix the 6 ESLint `set-state-in-effect` warnings or document the exception
- Clean up the 18 `.tmp/.nonulls/.clean` debris files and the two `.tsconfig.*.json` scratch files in the project root
- Remove dead exports in `lib/themes.ts` or import them
- Either delete the two re-export shims (`DashboardShell.tsx`, `PortfolioPreview.tsx`) or document them
- Resolve the “Sign in” placeholders — either implement them or replace with explicit “Demo · sign in disabled” UI
- Verify `next build` succeeds in a network-enabled environment

7.4 What should be tested

- Manual smoke test of all 12 routes
- Modal trap tests (Esc, click-outside, focus return)
- Save → unsave → save round-trip via `localStorage`
- Invite → notification → chat flow timing
- Refresh on each protected route confirms the hydration guard works

7.5 What should be refactored before scaling

- Replace `lib/candidates/data.ts` with a server-fetched paginated list (currently the marketplace renders all 17 every render)
- Extract a `<Modal>` primitive — modal logic (Esc, focus, scroll lock, backdrop click) is duplicated 4 times (`CandidateDetailModal`, `ContactForm.SuccessModal`, `ConfirmDialog`, `NotificationBell.ReachOutModal`)
- Centralize chip + score-bar components (each marketplace card/modal/profile reimplements them inline)
- Migrate forms to React Hook Form + Zod
- Consider TanStack Query when adding network calls — context-based caches won't scale to per-key invalidation

7.6 Security / privacy considerations

- `LocalStorage` stores potentially personal data (candidate name, target job, portfolio entries). On a shared machine this is recoverable. Once auth is in place, encrypt-at-rest is not required for `localStorage` but the team should at minimum clear it on logout.
- No XSS surface today (no `dangerouslySetInnerHTML` used; verified by grep).
- Once a real backend exists, candidate PII flows through the marketplace endpoints — apply standard data-minimization (don't ship email/phone in marketplace lists; only on invite accept).
- The public profile route (`/candidates/[id]`) is intentionally open — when this becomes real, gate it behind an `"is_public"` candidate flag and add `robots.txt` rules.

7.7 Performance

- 17 cards / 12 routes / no heavy assets. No measurable concern.
- `Parallax` and `FeaturesCarousel` use `rAF` + `prefers-reduced-motion` — good.
- `useGrowthLoop` runs a single timer chain — no leaks.

8. Prioritized Recommendations

Priority 1 — Critical fixes before production

#	Issue	Affected	Why	Suggested fix	Effort
1	No <code>error.tsx</code>	All routes	Uncaught error → Next default error page	Add <code>src/app/error.tsx</code> (App Router error boundary) and <code>src/app/global-error.tsx</code>	Low
2	No <code>not-found.tsx</code>	Dynamic routes	<code>notFound()</code> in three pages shows the default Next 404	Add <code>src/app/not-found.tsx</code> and a friendly empty state	Low

#	Issue	Affected	Why	Suggested fix	Effort
3	Candidate notifications lost on refresh	NotificationsContext	Real notifications need persistence; users will see disappearing alerts	Add localStorage hydration to Notification-Provider mirroring SavedCandidatesContext	Low
4	Marketplace looks real; mock data could deceive	lib/candidates, marketplace UI	Demo.ts, confusion / liability	Add a global "Demo data" banner or gate MOCK_CANDIDATES behind NEXT_PUBLIC_ENABLE MOCKS env flag	Low
5	"Sign in" placeholders in 3 spots	TopMenu, Employer-TopMenu, Navbar	Looks broken to users	Pick auth (NextAuth recommended) and wire one real endpoint; or display "Coming soon" toast	Medium
6	ESLint 6 errors (set-state-in-effect)	5 context providers	Failing lint blocks CI	Either refactor to React 19 useSyncExternalStore for localStorage hydration (cleanest), or add a file-scoped // eslint-disable-next-line react-hooks/set-state-in-effect with justification	Medium
7	Verify next build succeeds	n/a	Cannot deploy without build	Run npm install && npm run build on a developer machine with network access	Low

Priority 2 — Important before backend integration

#	Issue	Affected	Why	Suggested fix	Effort
1	All data in-browser only	All contexts	Cannot scale, sync, or share between devices	Define API contract (see §6.2), wire one endpoint at a time, treat localStorage as a draft cache	High
2	No form validation	ContactForm, RoleStep, SkillEvolutionCard	Spam / XSS / bad data	Adopt React Hook Form + Zod schemas	Medium
3	Modal logic duplicated 4 times	CandidateDetailModal, ContactForm.SuccessModal, ConfirmDialog, Notification-Bell.ReachOutModal	But only one are not fixed	Extract <Modal> primitive in src/components/ui/Modal.tsx	Medium
4	IntentContext.Provider is "candidate" \ null	src/lib/context/IntentContext	Cannot distinguish employer from candidate in shared logic	Widen to "candidate" \ "employer" \ null, gate routes accordingly	Low
5	Chat reply simulated client-side	ChatInterface	Will die hard during a real demo if backend is partial	Replace simulated reply with a true server round-trip; show typing indicator	Low
6	No types for "loading" / "errored" UI state on any context	5 providers	Cannot show partial / failed loads	Add status: 'idle' \ 'loading' \ 'ready' \ 'error' to each	Medium

Priority 3 — Polish / refactor / nice-to-have

#	Issue	Affected	Why	Suggested fix	Effort
1	18 leftover .tmp/.nonulls/.clean files	src/**	Repository hygiene, Grep noise	git rm the debris or add to .gitignore	Low

#	Issue	Affected	Why	Suggested fix	Effort
2	Dead exports in lib/themes.ts	appTheme, themes, semantic not imported	Code rot	Delete or import; if keeping the contract comment, move it into CLAUDE.md or globals.css	Low
3	Two re-export shims (DashboardShell.tsx, Portfolio-Pre-view.tsx)	Legacy	Confuses search	Delete (no callers found) or annotate with // kept for X and an ESLint-friendly comment	Low
4	Two empty .tsconfig.*.json files at root	Project root	Scrub debris	Delete	Low
5	MenuButtonStub src/components/MenuButtonStub “Engage with Employers” still labeled “soon”	src/components/MenuButtonStub	Missing employer flow now exists	Replace with an 	Low
6	“Sign in” link in Navbar jumps users into the candidate dashboard	src/components/Navbar	Confusing	Disable or route to a real sign-in modal	Low
7	Candidate scoring fields are static	lib/candidates	Looks like a black-box	When a backend exists, compute server-side; add a “How we score” affordance per CLAUDE.md philosophy	Medium
8	No prefers-reduced-motion opt-out on useGrowthLoop animation chain	Dashboard	Accessibility / motion sensitivity	Gate the timer cadence on window.matchMedia("(prefers-reduced-motion: reduce)").matches and run at 0 ms instead of 850 ms	Low

#	Issue	Affected	Why	Suggested fix	Effort
9	Score-bar / chip components duplicated inline	CandidateCard, Inconsistency risk CandidateDe-tailModal, SavedCandidateCard, candidate profile		Extract <code><ScoreBar></code> and <code><Chip></code> in <code>src/components/ui/</code>	Medium

9. Commands & Check Results

Command	Exit	Notes
<code>wc -l src/**/*.{ts,tsx}</code>	0	12,348 lines of TS / TSX in <code>src/</code>
<code>./node_modules/.bin/tsc --noEmit --pretty false</code>	1	Two errors in stale <code>.next/dev/types/{routes.d.ts, validate</code> Source is clean — confirmed by running <code>tsc</code> against a <code>src/-only</code> project file (<code>tsc -p .tsconfig.src.json</code> → exit 0).
<code>./node_modules/.bin/eslint src</code>	1	6 errors, all <code>react-hooks/set-state-in-effect</code> , all in context-hydration effects (see §5.6). 0 warnings.
<code>./node_modules/.bin/next build</code>	1	Fails offline — needs to download <code>@next/swc-*</code> from <code>registry.npmjs.org</code> . Confirmed expected by CLAUDE.md §7 (the sandbox has no internet).
Null-byte scrub (<code>tr -d '\000'</code>)	0	No null bytes remain in any source file

9.1 Suggested fixes

- **Stale dev artifacts:** `rm -rf .next/dev` (or `.next` entirely), then re-run `npm run dev` once to regenerate. The truncated `routes.d.ts` is a one-time artifact, not a recurring bug.
- **ESLint errors:** See Priority 1 recommendation #6. Quickest path is a top-of-file `/* eslint-disable react-hooks/set-state-in-effect */` with a comment pointing at the SSR-hydration rationale in CLAUDE.md §9.
- **next build:** Run on a developer machine (or CI with internet).

10. Appendix

10.1 Key files

File	Lines (≈)	Purpose
src/app/layout.tsx	60	Root providers stack (six contexts)
src/app/page.tsx	~520	Marketing landing
src/components/dashboard/useGrowthLoop.ts	155	The 8-step growth-loop state machine
src/lib/context/IntentContext.tsx	90	Candidate intent (localStorage)
src/lib/context/EmployerContext.tsx	110	Employer hiring goal (localStorage)
src/lib/context/SavedCandidatesContext.tsx	230	Saved + invited + notifications + chats
src/lib/candidates/data.ts	230	17 mock candidates
src/lib/jobs/destinations.ts	85	4 mock destinations + match helpers
src/components/employer/EmployerAppShell.tsx	60	Employer shell + redirect guard
src/components/marketplace/CandidateCard.tsx	180	Marketplace card with scores + skills + CTAs
src/components/marketplace/InviteForm.tsx	220	Invite composer + success modal
src/components/marketplace/ChatInterface.tsx	145	Private chat surface
src/app/globals.css	~350	Design tokens, glass utilities, animations
CLAUDE.md	<200	Project operating manual (per repo conventions)

10.2 Routes summary

12 page routes, 3 dynamic. No API routes.

10.3 Contexts summary

Context	Persisted?	Storage key
IntentContext	Yes	career-os-intent
NotificationsContext	No (in-memory only)	—
PortfolioContext (via usePortfolio)	Yes	career-os-portfolio
ChaptersContext	Yes	career-os-events
EmployerContext	Yes	career-os-employer-goal, career-os-employer-onboarded
SavedCandidatesContext	Yes	career-os-employer-{saved,invited,notifications,chats}

10.4 Assumptions made in this report

- The 17 candidate records in lib/candidates/data.ts are intentional mocks (not stale demo data left in by accident).
- “Sign in” placeholders are intentional per the prototype spec.
- The two .tsconfig.*.json empty files at the project root are scratch files I could not delete due to the workspace permission layer — they are valid empty JSON ({}) and ignored by tooling.

- The two `.next/dev/types/*` files with syntax errors are stale Next.js dev artifacts; a fresh `next dev/next build` will regenerate them. I could not regenerate them in this sandbox because of the offline SWC issue.
- The 18 `*.nonulls/*.clean/*.tmp` files in `src/` are intentional artifacts of the workflow documented in CLAUDE.md §8, not files awaiting use.

10.5 What I could not verify

- `next build` (offline sandbox, no network)
- Production runtime behavior (no `next start`)
- Cross-browser behavior (no headless browser available)
- Lighthouse / Core Web Vitals
- Real device responsive testing
- Exact color contrast ratios at all token combinations

End of report.